Linguistic Misinformation Markers

Jennifer Flake, Nicole Shantz, Shiao-li Green, Rachelle De Jager

Abstract

1 Introduction

We aim to build an annotated corpus that captures linguistic patterns distinguishing misinformation, opinion/editorial, from verified news. Our goal is to produce a dataset resource useful for researchers studying how language signals credibility — or the lack of it. We plan to enhance the Twitter Misinformation Dataset that is currently available on Hugging Face with the goal of contributing the enhanced dataset to Hugging Face.

Not all social media posts and news items are completely composed of fact or completely composed of misinformation. For example, a newspaper editorial likely contains facts, as well as opinions. Some social media posts are all about emotion without any facts, yet also without any misinformation.

Questions:

1. What linguistic markers distinguish misinformation from factual/verified information in news articles, news headlines and social media posts?

2. Is there a way to distinguish a third category of information?

Hypothesis: News and social media texts will have measurable differences in linguistic features — such as higher use of superlatives, emotional language, capitalization, vague sourcing, and clickbait phrasing, depending on whether the text aims to disseminate factual information, spread misinformation, or express emotions and opinions.

2 Data

We used the Hugging Face Twitter Misinformation Dataset (Minassian) and annotated 1,000 examples labeled with the following categories:

1. `adjectives`: Adjectives with suffixes -ful, -less, -ment, -ness, -ing, or -ible.

2. `all_caps`: Capitalised words that are not acronyms.

3. `exclamation_marks`: Any number of exclamation marks.

4. `hedging`: Words found in the hedging lexicon (hedging lrec).

5. `unk`: Profanity.

We removed duplicate text entries and also considered texts to be duplicates when only the URL was modified, likely due to tokens added when choosing to share to Twitter/X from a third party website.

To ensure the hand-annotated dataset was manageable, we limited text length to 560 characters (twice that of a tweet); however, we believe that our models can scale to longer text.

They are currently labeled with 0 (factual) or 1 (misinformation), however, we aim to expand this to 3 classes, incorporating opinion. Given this, we added a column for opinion, with 1 as opinion as 0 as not opinion. Opinion was determined through first person point of view texts, not headlines that conveyed opinion, that showed a stance or preference towards a circumstance.

Given the small size of the dataset at 1000 examples, we decided on a 60-20-20 train-validation-test split. This was conducted using stratification, to ensure proportions in each split reflect the main dataset in terms of distribution of misinformation (35% misinformation, 65% factual) and opinion (41% opinion, 59% not opinion). We also included each main annotator as a stratification factor, being annotators that individually represented at least 20% of all annotations (Jennifer, Nicole, Shiao-li, and Rachelle). The remaining three annotators (AI, Majority Vote, and Jasmine) were not included in the stratification to avoid imbalance. To ensure reproducibility, the dataset splits are added as a

column to the main dataset and the script sets a random seed to 344.

For sprint 4, we used `df.sample` to select 250 additional rows from the cleaned dataset, still limiting to 560-character text limits and no exact match/duplicates in text before the URL.

3 Methodology

3.1 Baseline Models

3.1.1 Traditional Method

For our traditional model, we used logistic regression. This model is appropriate for the text classification task due to its scalability for sparse features and training speed.

To evaluate the model's ability to identify opinions, text and numeric features were used. TF-IDF vectorization was applied over unigrams and bigrams for text data assessment. Additionally, our annotated linguistic features were scaled using StandardScaler before combining them with the TF-IDF matrix. The TF-IDF captures vocabulary and content patterns, while the annotated linguistic features identify writing style markers. Therefore, the inclusion of both feature types was necessary to adequately measure the differences between factual information, misinformation, and personal opinions. To account for imbalances between these classes, the model was trained with balanced class weights.

For hyperparameter 'C', we used GridSearchCV to conduct 5-fold cross-validation on the training set. We used the macro F1 score to select the appropriate C value to account for class imbalances.

The results of the development set indicate that misinformation (Acc = 0.930; F1 = 0.905; AUC-ROC = 0.962) showed higher classification scores compared to opinions (Acc = 0.690; F1 = 0.683; AUC-ROC = 0.753).

Our error analysis suggested that the model encountered issues with misidentifying short emotional texts (false positives) and mislabeled fact-based opinion texts (false negatives).

3.1.2 Neural Method

For the neural model, a Convolutional Neural Network (CNN) was used in conjunction with NLTK's tweetTokenizer and FastText frozen embeddings. Since the example texts in the corpus are of short length, opinion signals may exist in 3-gram, 4-gram, or 5-gram sequences, rather than in longer sequences. CNN was chosen because it relies on local markers by using a shifting window of n-grams to find patterns in text. In addition, given the low resource constraints of the project, CNN is robust against overfitting. Hyperparameters were chosen according to recommendations in "Convolutional Neural Networks for Sentence Classification," (Kim, 2014) which recommends filter windows (h) of 3, 4, 5 with 100 feature maps each, dropout rate (p) of 0.5, and l2 constraint (s) of 3.

Both the tokenizer and word embeddings were chosen because of their specialization in transforming internet text. NLTK's tweetTokenizer was used because it was developed to handle text boundaries that are unique to Twitter, including hashtags and emojis. Finally, FastText embeddings were selected because they are robust against typos and slang by using subword n-grams. The FastText embeddings were frozen to prevent additional overfitting of the training data, since the training set only included 600 examples.

When the CNN neural method we used was evaluated on the development data set, it scored Macro F1 = 0.7261, F1(not opinion) = 0.7589, F1(opinion) = 0.6932 and AUC-ROC= 0.7930.

During Sprint 2, we modified the original baseline_cnn.py file to include the ability to train the model on a different target. After making this change, we used the same code to train a misinformation classifier separately. When we used the development data set to evaluate the model trained with the misinformation target, it scored Macro F1 = 0.8773, F1 (not misinformation) = 0.9356, F1(misinformation) = 0.8190, and AUC-ROC = 0.9528.

4 Experimentation

4.1 Ensembling

4.1.1 Simple Ensemble

We first implemented a simple ensembling via soft-voting where each baseline model's predicted probabilities are averaged equally, then applied a decision threshold of 0.5 for the final averaged prediction. This preserved model confidence in its class prediction rather than relying on hard-voting.

4.1.2 Motivated Ensemble

This ensemble method was selected to address class imbalances and differences in model performance. The predicted probabilities were weighted by the Macro F1 score and the decision thresholds were tested with 0.05 steps from 0.3 to 0.7.

The threshold remained 0.50 for opinion classification, resulting in a slight improvement (Macro F1 = 0.7431) over the previous ensembling method (Macro F1 = 0.7186). The misinformation classification used a 0.40 threshold and showed a marginal decline (Macro F1 = 0.9180) from the simple ensembling method (Macro F1 = 0.9230).

4.2 Transfer Learning

4.2.1 Traditional Method

We decided to augment the data by applying the same embeddings as the Neural Method - FastText frozen embeddings.

This involved taking the embeddings-related functions from `cnn_baseline.py` adjusting the return from a tensor to a numpy array and creating an additional function to get the document embeddings, followed by converting from dense to sparse matrix to feed into logistic regression.

It is worth noting that embeddings typically perform better in non-linear models like neural networks. To balance the consideration of TF-IDF and embeddings to reduce feature imbalance, TF-IDF weighted pooling was applied to give higher weight of embeddings to more important words in corpus.

This resulted in minimal improvement from the Logistic Regression baseline, with a slightly lower score for F1 of not opinion and still did not outperform the baseline CNN approach.

4.2.2 Neural Method

Since our baseline model already included transfer learning via FastText frozen embeddings, we implemented an additional transfer learning signal by implementing a model that uses a single convolutional backbone across both misinformation and opinion classification tasks. The backbone is trained on the text of each example to find patterns that are jointly useful for predicting opinion and misinformation, rather than optimizing for either label in isolation. The signal from one task provides secondary supervision to the other. In a shared backbone, the model runs every example of the dataset through the same convolutional layers. The layers produce a single vector for each example text. Then, the resulting vector is passed to two separate output layers to predict opinion and misinformation.

The original baseline_cnn.py file was modified to support multiple or individual training targets. Specifically, the train_model() function was altered to include an optional parameter, train_targets, which supports the specification of an individual target or multiple targets in the form of a python list. If multiple targets are selected, they will be trained with the shared convolutional backbone.

According to our metrics, this approach actually reduced the performance for both the classification of misinformation and opinion. The resulting metrics for opinion classification are: Macro F1 = 0.7090, F1(not opinion) = 0.7264, F1(opinion) = 0.6915, and AUC-ROC= 0.7707. The resulting metrics for misinformation classification are: Macro F1 = 0.8667, F1(not misinformation) = 0.9333, F1(misinformation) = 0.8000, and AUC-ROC = 0.9528.

4.3 Multi-Task Learning

We used Part-of-speech (POS) tagging to enhance our models through Multi-Task learning. POS tagging was selected because opinion tends to feature more pronouns, adjectives, and adverbs than factual reporting. Factual reporting relies more on nouns and verbs. Since we are also training models for misinformation, we wanted to investigate whether our misinformation models, which were already robust, would also improve with POS tagging.

5 Results

5.1 Metrics

Macro F1 was included since there is some class imbalance, this metric weights each class equally.

F1 (opinion) and F1 (not opinion) these are the components of Macro F1 and are helpful to report separately as they indicate which class the models are better at predicting.

F-beta score at beta=1.5 was added to penalize false negatives, motivated by our intuition that incorrectly tagging text as factual when it is actually misinformation carries greater harm, however not sufficient harm to warrant beta=2.

AUC-ROC score is reported as a secondary metric to assess the model's ranking of text examples by confidence, independent of the decision threshold of 0.5.

6 Detailed Analysis

We have been creating, modifying and comparing models for two different targets: opinion and misinformation. All of our models are based on CNN, Logistic Regression, or an ensemble with both types. One of the interesting results is that different models perform better on different tasks. All

Model	Macro-F1	F1.5	AUC
TextCNN MTL (POS)	0.7362	0.7203	0.7913
Lean Motivated Ensemble	0.7347	0.7666	0.7989
Motivated Ensemble	0.7299	0.7698	0.7967
Soft Vote Ensemble	0.7267	0.7152	0.7957
TextCNN	0.7261	0.7087	0.7930
Lean Soft Vote	0.7177	0.7165	0.7985
TextCNN MTL (POS + linguistic)	0.7172	0.7101	0.7917
LogReg (+ embeddings)	0.7023	0.6962	0.7698
TextCNN Transfer	0.6963	0.6806	0.7701
LogReg MTL (cascaded POS)	0.6963	0.6806	0.7486
LogReg	0.6931	0.6530	0.7514

Table 1: Model comparison for opinion

Model	Macro-F1	F1.5	AUC
Lean Soft Vote	0.9161	0.8794	0.9676
Lean Motivated Ensemble	0.9161	0.8794	0.9676
Motivated Ensemble	0.9102	0.8743	0.9687
Soft Vote Ensemble	0.9031	0.8603	0.9689
LogReg (+ embeddings)	0.8889	0.8318	0.9501
LogReg MTL (cascaded POS)	0.8831	0.8269	0.9157
LogReg	0.8784	0.7975	0.9630
TextCNN	0.8773	0.8221	0.9528
TextCNN MTL (POS)	0.8758	0.8125	0.9608
TextCNN Transfer	0.8742	0.8027	0.9593
TextCNN MTL (POS + linguistic)	0.8627	0.7932	0.9489

Table 2: Model comparison for misinformation

Logistic Regression models outperform all CNN models on the misinformation classification task. However, all CNN models outperform all Logistic Regression models on the opinion classification task, with one exception. The exception is the TextCNN Transfer model, which was tied with LogReg MTL and surpassed by LogReg (+ embeddings). Overall, ensembles help classification in both tasks.

With a highest Macro F1 score at 0.7362, the opinion classification task appears to be the more difficult task. According to this metric, TextCNN MTL (POS) outperformed all of the other models. However, Lean Motivated Ensemble was only slightly behind it with a score of 0.7347. Notably, TextCNN MTL (POS + linguistic) did not perform as well as TextCNN MTL (POS). This suggests that our annotated linguistic markers weaken the learning signal for that model.

Examples of false positives for our best opinion classifier suggest that the model is potentially using pronouns as a signal for opinion. Most of the eight shown examples feature casual writing and use of pronouns. Examples, like [17] 'Birds in a blizzard. We put out extra sunflower seeds since their usual food sources just got...' are personal observations and should not be labeled as 'opinion.' Another example of features that may send a noisy signal is when all capital letters and exclamation points are used in factual statements. In the text, [209] 'OMG! Gas prices affected by the hurricane!! Up 3 cents last night, 5 cents more before tomorrow!!!... oh wait, I ride a bike... nevermind...', the model likely sees the exclamation marks, use of all capital letters, and the use of the pronoun "I" as a signal of "opinion." Superficial markers of emphasis may be features that correlate with opinion, but they may also be used in non-opinion writing as well.

Overall, in our best opinion classification model, there were 21 cases of false negatives out of a total of 200 examples. However, there are some cases in the examples printed in this notebook that cause us to question the use and distinction of the opinion label overall. In the example, "A total of 44 people have been killed in California's devastating wildfire as authorities continue to find bodies in burnt-out cars and homes," We may agree with the model and not the "opinion" label. Different annotators may have different thoughts on what constitutes opinion. Therefore, it is understandable why a model may disagree with the opinion label at times. Other examples of false negatives include adjectives that the model may not read as indicating opinion, but our annotators did. Our annotators marked "Intense Footage Of A Helicopter Fighting A Wildfire Feet Away From Cars In California" as opinion, likely due to the use of "intense." However, the model classified it as not opinion.

In the misinformation classification task, Lean Soft Vote and Lean Motivated Ensemble performed best by combining the two best performing models: LogReg (+ embeddings) and TextCNN MTL (POS). Overall, the models have consistently performed well on the misinformation task. Progress was made by limiting the number of models in the ensembles to include only 2, rather than all models created to date. Using all models in an ensemble may weaken the learning signal when weaker models are contributing to the ensemble. Lean Soft Vote and Lean Motivated Ensemble scored the exact same score: Macro F1 = 0.9161. These models performed so well that looking at examples of false positives and false negatives may yield limited insights. Two of the examples of false negatives suggest annotation errors, rather than model errors. A nursing job posting [565] and a quote with an attribution [559] are both classified as not misinformation by the model, but annotated as misinformation. On those two examples, we agree with the model's classification. LogReg had a Macro F1 of 0.8784

and TextCNN had a Macro F1 of 0.8773. Improving the Macro F1 to 0.9161 with either of the two lean ensembles that use both TextCNN MTL (POS) and LogReg (+ embeddings) represents improvement to our misinformation classification.

Since our best Macro F1 score for misinformation classification is 0.9161 and our best for opinion classification is 0.7362, there is a significant gap in the models' ability to classify misinformation and opinion. Beyond the models themselves, there may be corpus-related reasons for this gap. Perhaps the task of annotating opinion is inherently more difficult or subjective and, therefore, more likely to have different annotations for the same text. Perhaps the text in our corpus is particularly difficult to annotate for opinion. Hopefully, all of the models will improve with more training data.

7 Conclusion

8 Limitations

9 Ethical Considerations

References

hedging lrec. Hedge words. https://github.com/hedging-lrec/resources/blob/master/hedge_words.txt.

Yoon Kim. 2014. Convolutional neural networks for sentence classification. *Preprint*, arXiv:1408.5882.

Roupen Minassian. Twitter misinformation dataset. https://huggingface.co/datasets/roupenminassian/twitter-misinformation.